



Objetivo

Identificar y evitar los antipatrones más comunes en el desarrollo de software.

¿Qué son los antipatrones?

Los **antipatrones** son soluciones aparentemente obvias a problemas recurrentes que generan **más problemas** de los que resuelven.

Características de un antipatrón

- Parece una **buena idea** al principio
- Se usa **frecuentemente**
- Genera **problemas** a largo plazo
- Es **difícil de cambiar** una vez implementado

Antipatrones de código

1. Spaghetti Code

Problema

Código sin estructura clara, con dependencias enredadas.

```
// Función que hace demasiado
function processUser(userData: any): any {
  let result = userData;

  // Validación mezclada con lógica de negocio
  if (!result.email || !result.email.includes('@')) {
```

```

        console.log('Invalid email');
        result.errors = result.errors || [];
        result.errors.push('email');
    }

    // Transformación de datos
    result.name = result.firstName + ' ' + result.lastName;
    result.age = new Date().getFullYear() - new
Date(result.birthDate).getFullYear();

    // Persistencia
    if (result.errors && result.errors.length > 0) {
        console.log('User has errors, not saving');
    } else {
        console.log('Saving user to database...');
        // Lógica de base de datos aquí
        result.id = Math.random().toString();

        // Notificaciones
        console.log(`Sending welcome email to ${result.email}`);

        // Logging
        console.log(`User ${result.name} created with ID ${result.id}`);
    }

    return result;
}

```

☑ Solución: Separación de responsabilidades

```

interface User {
    id?: string;
    firstName: string;
    lastName: string;
    email: string;
    birthDate: string;
}

interface ValidationResult {
    isValid: boolean;
    errors: string[];
}

// Separar responsabilidades
class UserValidator {
    validate(userData: Partial<User>): ValidationResult {
        const errors: string[] = [];

        if (!userData.email?.includes('@')) {
            errors.push('Invalid email format');
        }
    }
}

```

```

    return {
      isValid: errors.length === 0,
      errors
    };
  }
}

class UserTransformer {
  transform(userData: Partial<User>): User {
    return {
      ...userData,
      id: Math.random().toString(),
      firstName: userData.firstName || '',
      lastName: userData.lastName || '',
      email: userData.email || '',
      birthDate: userData.birthDate || ''
    };
  }
}

class UserService {
  constructor(
    private validator: UserValidator,
    private transformer: UserTransformer,
    private repository: UserRepository,
    private notificationService: NotificationService
  ) {}

  async createUser(userData: Partial<User>): Promise<User | null> {
    const validation = this.validator.validate(userData);

    if (!validation.isValid) {
      console.log('Validation errors:', validation.errors);
      return null;
    }

    const user = this.transformer.transform(userData);
    const savedUser = await this.repository.save(user);

    await this.notificationService.sendWelcomeEmail(savedUser.email);

    return savedUser;
  }
}

```

2. 🏠 God Object

✖ Problema

Una clase que hace demasiado.

```

class UserManager {
    // Gestión de usuarios
    createUser(userData: any): User { /* ... */ }
    updateUser(id: string, data: any): User { /* ... */ }
    deleteUser(id: string): void { /* ... */ }

    // Autenticación
    login(email: string, password: string): string { /* ... */ }
    logout(token: string): void { /* ... */ }
    validateToken(token: string): boolean { /* ... */ }

    // Permisos
    hasPermission(userId: string, permission: string): boolean { /* ... */ }
    grantPermission(userId: string, permission: string): void { /* ... */ }

    // Notificaciones
    sendEmail(userId: string, subject: string, body: string): void { /* ... */ }
    sendSMS(userId: string, message: string): void { /* ... */ }

    // Reportes
    generateUserReport(): string { /* ... */ }
    exportUsers(): string { /* ... */ }

    // Configuración
    updateSettings(settings: any): void { /* ... */ }
    getSettings(): any { /* ... */ }
}

```

☒ Solución: Separar responsabilidades

```

// Cada clase tiene una responsabilidad clara
class UserRepository {
    create(userData: User): User { /* ... */ }
    update(id: string, data: Partial<User>): User { /* ... */ }
    delete(id: string): void { /* ... */ }
    findById(id: string): User | null { /* ... */ }
}

class AuthenticationService {
    login(email: string, password: string): string { /* ... */ }
    logout(token: string): void { /* ... */ }
    validateToken(token: string): boolean { /* ... */ }
}

class PermissionService {
    hasPermission(userId: string, permission: string): boolean { /* ... */ }
    grantPermission(userId: string, permission: string): void { /* ... */ }
}

class NotificationService {
    sendEmail(userId: string, subject: string, body: string): void { /* ... */ }
}

```

```

    sendSMS(userId: string, message: string): void { /* ... */ }
}

class ReportService {
    generateUserReport(): string { /* ... */ }
    exportUsers(): string { /* ... */ }
}

```

3. 🔗 Tight Coupling

✖ Problema

Clases fuertemente acopladas.

```

class EmailService {
    sendEmail(to: string, subject: string, body: string): void {
        console.log(`Sending email to ${to}: ${subject}`);
    }
}

class UserService {
    private emailService = new EmailService(); // Acoplamiento fuerte

    registerUser(user: User): void {
        // Lógica de registro
        console.log('User registered');

        // Dependencia directa
        this.emailService.sendEmail(
            user.email,
            'Welcome!',
            'Welcome to our platform!'
        );
    }
}

```

☑ Solución: Inyección de dependencias

```

interface NotificationService {
    notify(to: string, message: string): void;
}

class EmailService implements NotificationService {
    notify(to: string, message: string): void {
        console.log(`Email to ${to}: ${message}`);
    }
}

class SMSService implements NotificationService {
    notify(to: string, message: string): void {

```

```

        console.log(`SMS to ${to}: ${message}`);
    }
}

class UserService {
    constructor(private notificationService: NotificationService) {}

    registerUser(user: User): void {
        console.log('User registered');
        this.notificationService.notify(user.email, 'Welcome to our platform!');
    }
}

// Uso flexible
const userService = new UserService(new EmailService());
// o
const userServiceWithSMS = new UserService(new SMSService());

```

4. 🛠 Golden Hammer

✖ Problema

Usar la misma solución para todo.

```

// Usar clases para todo, incluso cuando no es necesario
class MathUtils {
    add(a: number, b: number): number {
        return a + b;
    }

    multiply(a: number, b: number): number {
        return a * b;
    }
}

class StringUtils {
    capitalize(str: string): string {
        return str.charAt(0).toUpperCase() + str.slice(1);
    }

    reverse(str: string): string {
        return str.split('').reverse().join('');
    }
}

// Uso innecesariamente complejo
const mathUtils = new MathUtils();
const result = mathUtils.add(2, 3);

```

☑ Solución: Herramienta apropiada para cada problema

```
// Funciones simples para operaciones sin estado
export const add = (a: number, b: number): number => a + b;
export const multiply = (a: number, b: number): number => a * b;

export const capitalize = (str: string): string =>
  str.charAt(0).toUpperCase() + str.slice(1);

export const reverse = (str: string): string =>
  str.split('').reverse().join('');

// Clases solo cuando hay estado o comportamiento complejo
class Calculator {
  private history: number[] = [];

  add(a: number, b: number): number {
    const result = a + b;
    this.history.push(result);
    return result;
  }

  getHistory(): number[] {
    return [...this.history];
  }
}
```

Resumen

Los antipatrones son **trampas comunes** en el desarrollo:

- **Reconócelos** temprano
- **Refactoriza** gradualmente
- **Prevén** con buenas prácticas
- **Aprende** de los errores

Recuerda: Es mejor prevenir antipatrones que tener que refactorizarlos después.